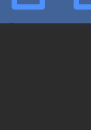


Quick Sort

- ✔ **Summary (Key Points First)**
 - **Quick Sort** is a **Divide & Conquer** algorithm that works in-place [no extra array like merge sort].
 - **Pivot selection** is key: choose any element [commonly first, last, random, or median].
 - Core idea:
 1. Pick a pivot.
 2. Partition the array: elements < pivot go left, > pivot go right.
 3. Recursively sort the left and right partitions.
 - **Time Complexity:**
 - **Best/Average Case:** $O(N \log N)$
 - **Worst Case** [already sorted or reverse sorted input with poor pivot choice]: $O(N^2)$
 - **Space Complexity:**
 - $O(1)$ for array [in-place], but **$O(\log N)$** auxiliary stack space due to recursion.
 - **Stable?** ✗ No (relative order of equal elements may change).
 - **In-place?** ✔ Yes.

Quick Sort Analysis

Characteristic	Description
 Algorithm Type	Divide & Conquer
 Pivot Selection	Key to performance
 Time Complexity (Best/Average)	$O(N \log N)$
 Time Complexity (Worst)	$O(N^2)$
 Space Complexity	$O(\log N)$
 Stable?	No
 In-place?	Yes

💡 Intuition

Quick Sort sorts by selecting a **pivot** and rearranging the array so that:

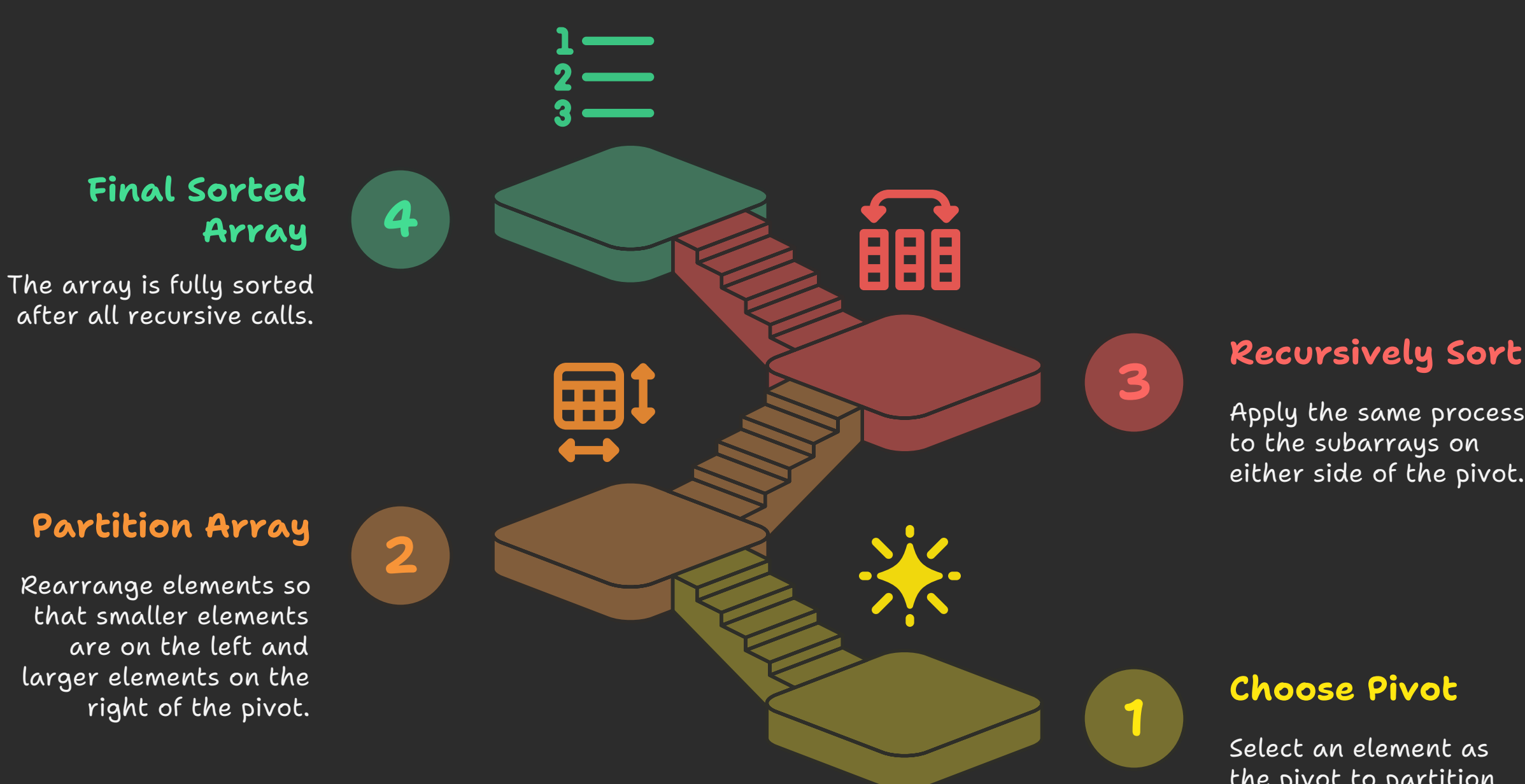
- All elements **less than the pivot** go to its left.
- All elements **greater than the pivot** go to its right. This process is recursively applied on subarrays formed on either side of the pivot.

For example, in array: {4, 6, 2, 5, 7, 9, 1, 3}, choosing 4 as pivot:→ Final position of 4 after partitioning is index 3→ Now left subarray: {3, 2, 1}, right subarray: {6, 5, 7, 9}→ Recursively apply the same process.

⚙️ Approach

- Use two pointers: low and high to represent subarray range.
- The **partition function** places the pivot in its final sorted position and returns the **partition index**.
- Recursively call quickSort on:
 - Left subarray: low to pivotIndex - 1
 - Right subarray: pivotIndex + 1 to high

Steps to Quick Sort



📝 Pseudocode

quickSort(arr, low, high):

```
if low < high:
    pivotIndex = partition(arr, low, high)
    quickSort(arr, low, pivotIndex - 1)
    quickSort(arr, pivotIndex + 1, high)
```

partition(arr, low, high):

```
pivot = arr[low]
i = low
j = high

while i < j:
    move i forward while arr[i] <= pivot
    move j backward while arr[j] > pivot
    if i < j: swap arr[i], arr[j]

swap arr[low] and arr[j]
return j (pivot's final position)
```

💻 Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[low];
    int i = low, j = high;

    while (i < j) {
        while (arr[i] <= pivot && i <= high - 1) i++;
        while (arr[j] > pivot && j >= low + 1) j--;
        if (i < j) swap(arr[i], arr[j]);
    }

    swap(arr[low], arr[j]);
    return j;
}

void qs(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pIndex = partition(arr, low, high);
        qs(arr, low, pIndex - 1);
        qs(arr, pIndex + 1, high);
    }
}

vector<int> quickSort(vector<int> arr) {
    qs(arr, 0, arr.size() - 1);
    return arr;
}

int main() {
    vector<int> arr = {4, 6, 2, 5, 7, 9, 1, 3};
    cout << "Before quick sort:\n";
    for (int i : arr) cout << i << " ";
    cout << endl;

    arr = quickSort(arr);
    cout << "After quick sort:\n";
    for (int i : arr) cout << i << " ";
    cout << endl;

    return 0;
}
```

📊 Complexity Analysis

- **Best / Average Case:**
 - Recurrence: $T[n] = 2T[n/2] + O[n] \rightarrow$
 - **Time Complexity:** $O(N \log N)$
- **Worst Case** [pivot is min or max]:
 - Recurrence: $T[n] = T[0] + T[n-1] \rightarrow$
 - **Time Complexity:** $O(N^2)$
- **Space Complexity:**
 - In-place sorting $\rightarrow O(1)$
 - Recursion stack $\rightarrow O(\log N)$ (best), $O(N)$ (worst)

Let me know if you want a dry run or diagrammatic flow for better clarity!